

K-NN, Unsupervised learning and Other Approaches [IIA – Lect._____]

Alessio Micheli

micheli@di.unipi.it



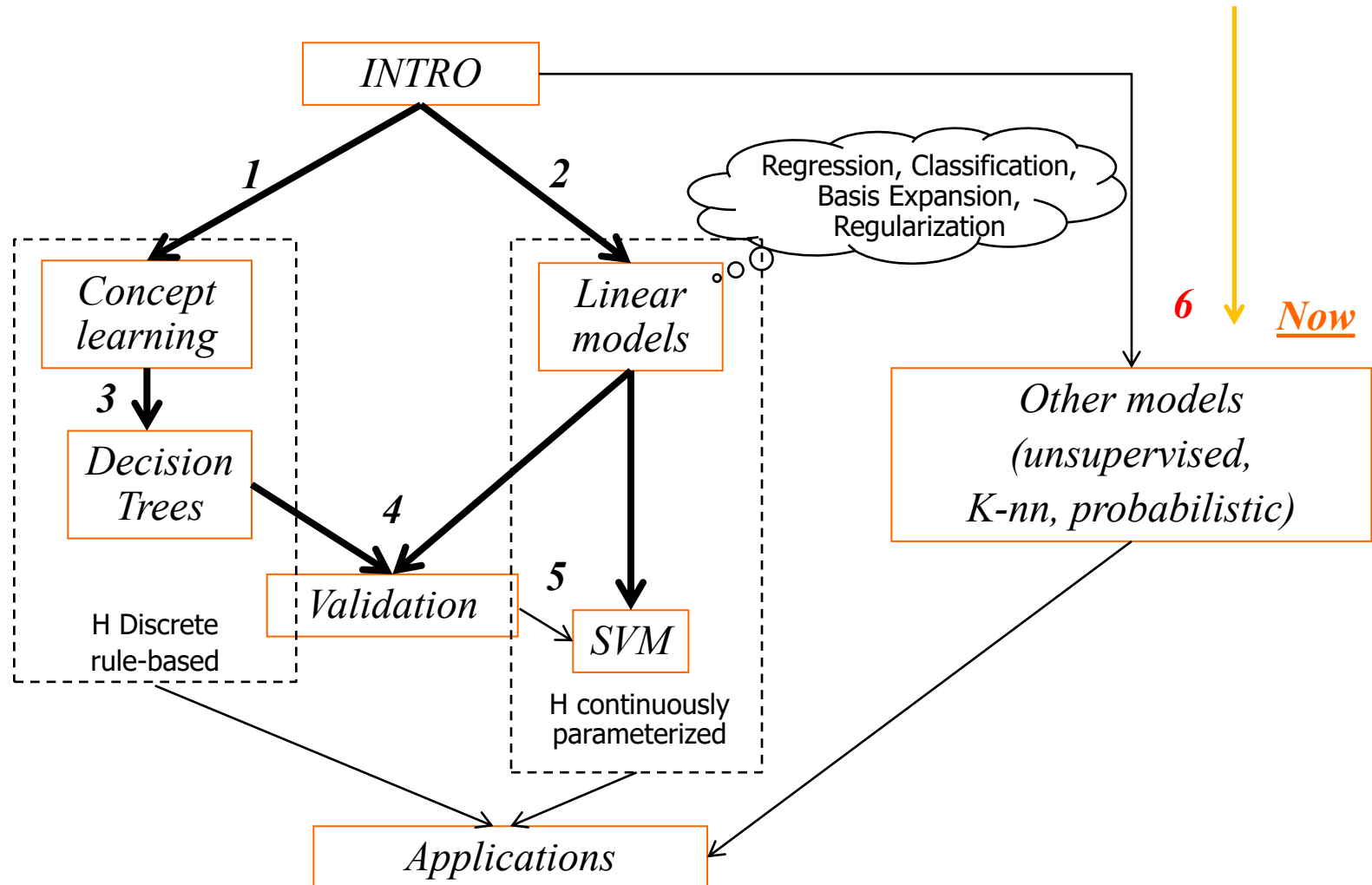
Dipartimento di Informatica
Università di Pisa - Italy

**Computational Intelligence &
Machine Learning Group**

DRAFT, please do not circulate! 2026

In the course flow

In the course: almost completed ... now the other models (a short survey)



AIM

- This introductory course cannot cover all the aspects of the large area of Machine Learning
- We left a series of paradigms, models, and algorithms
- Let's us have a look to some important (not seen so far) cases in the
 - Instance based approaches
 - Unsupervised learning tasks
 - Other ... (including Neural Networks)

An overview (for supervised learning)

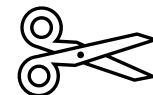


Dip. Informatica
University of Pisa

- Discrete spaces:
 - e.g. [in IIA] Conjunction of literals, Decision trees (propositional rules)
 - Inductive grammars, Inductive Logic Programming, ...
- Continuous spaces:
 - E.g. [in IIA] Multiple Linear Regression, Linear threshold unit (LTU), polynomial regression, ..., k-means [an intro in this lecture]
 - SVM [intro in IIA], Neural Networks [#ML] & Deep Learning [#ML]
- Probabilistic/Generative [an intro in NEXT lectures]**
 - Traditional parametric models: density estimation (e.g. coin toss), discriminant analysis, polynomial regression, ...
 - Graphical models: Bayesian networks, Naïve Bayes [in IIA]**, Markov models, Hidden Markov models, ...[#ML]
- Instance-based
 - Nearest neighbor [an intro in this lecture]



**** Not this year**
Folder ML-Bayesian Learning
come materiale facoltativo



K-NN

K- Nearest Neighbors (**supervised learning**)

1-Nearest Neighbor

Defs

- Simply store the training data $\langle \mathbf{x}_p, y_p \rangle$ $p=1 \dots l$
- Given an input $\mathbf{x}$ (with dimension n)
- Find the nearest training example $\mathbf{x}_i$
 - Find i s.t. we have $\min d(\mathbf{x}, \mathbf{x}_i) \rightarrow i(\mathbf{x}) = \arg \min_p d(\mathbf{x}, \mathbf{x}_p)$

– E.g. Euclidian distance :

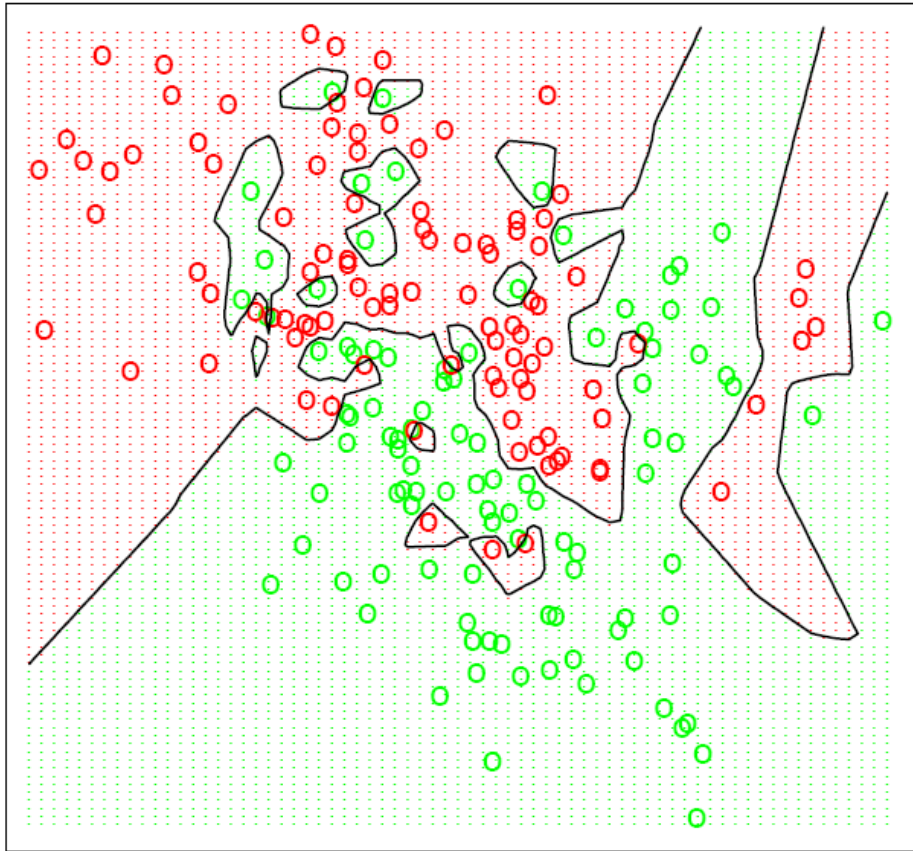
$$d(\mathbf{x}, \mathbf{x}_p) = \sqrt{\sum_{t=1}^n (x_t - x_{p,t})^2} = \|\mathbf{x} - \mathbf{x}_p\|$$

$\downarrow$
 Pattern $\mathbf{x}_p$, component t

- Then output y_i

1-nn

1-Nearest Neighbor Classifier

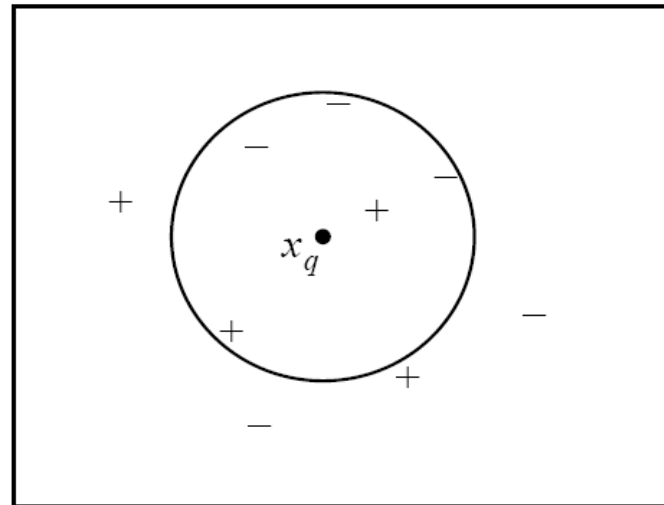


GREEN = 0, RED = 1

- Very flexible !
- No misclassifications in TR data: 0 training error: what for test data ?
- Decision boundaries is *not* linear: but it is quite *irregular*
- May be unnecessary noisy
- and prone to overfitting
- Remember also the case of SVM with RBF and small sigma value: “the classifier replies with the class of the closest point”

1-nn vs 5-nn example

- 1-nn return + for x_q
- 5-nn return – for x_q



Smoothing over a set of neighbors for noisy data

K-Nearest Neighbors

- A natural way to classify a new point is to have a look at its neighbors,

- and take an average *: $avg_k(\mathbf{x}) = 1/k \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} y_i$

Def

- where $N_k(\mathbf{x})$ is a neighborhood of $\mathbf{x}$ that contains exactly k neighbors (closest patterns according to *distance* d):
 - k-nearest neighborhood: **K-nn**.
- If there is a clear dominance of one of the classes in the neighborhood of an observation $\mathbf{x}$, then it is likely that the observation itself would belong to that class, too. Thus the classification rule is the **majority voting** among the members of $N_k(\mathbf{x})$. As before,

$$h(\mathbf{x}) = \begin{cases} 1 & \text{if } avg_k(\mathbf{x}) > 0.5 \\ 0 & \text{otherwise} \end{cases} \quad \text{for } y_i = \{0, 1\} \text{ (targets)}$$

- For **regression** task: use directly the *avg*: mean over K-nn

15-nn

15-Nearest Neighbor Classifier

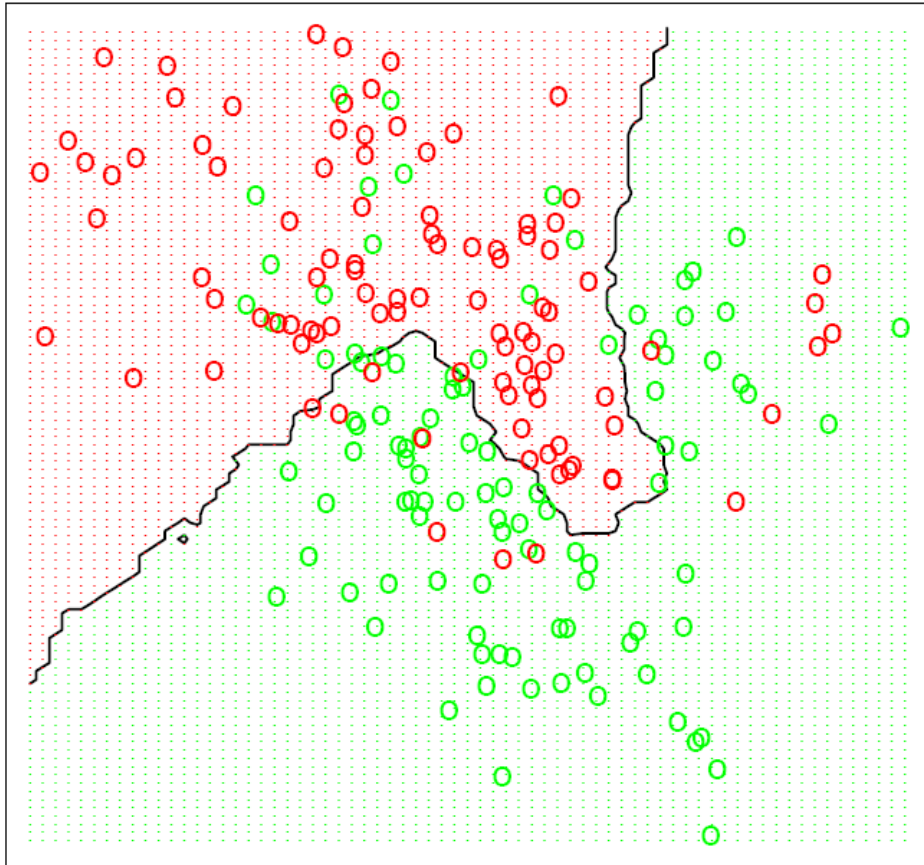


Figure 2.2:

The same classification example in two dimensions GREEN = 0, RED = 1

The predicted class is hence chosen by majority vote amongst the 15-nearest neighbors.

- Still very flexible !
- Some misclassifications in TR data
- Decision boundaries is *not* linear: it is still quite, although less, *irregular*
- Decision boundary adapts to the local densities of the classes

“Complexity” of K-NN

- Again... there is trade-off between underfitting and overfitting by the possible values of K

Classic U -shape curve of the test error moving between a

- extremely flexible case ($K=1$)
- up to a very rigid model ($K=l$): 1 mean for all the data (where did you see it?*)

K-nn for multi-class (#tech)

- Return the class most common amongst its k nearest neighbors

$$h(\mathbf{x}) = \arg \max_v \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} \mathbf{1}_{v, y_i}$$

$$\mathbf{1}_{v, y_i} = \begin{cases} 1 & \text{if } v = y_i \\ 0 & \text{otherwise} \end{cases}$$

K-nn: An extreme

- Not a global hypothesis for all the instances $\rightarrow$ no model to be fit
 - We need to memorize all the input examples
 - Non parametric model (in contrast think to the parametric linear model with a set of fixed size parameters $\mathbf{w}$)
- *Local estimations* (by locally constant functions) *vs* *global linear* approximation/estimation of the target function (over the instance space)

Def.

Lazy, memory based, instance-based, **distance-based methods**

(some) Limits of K-nn: computational cost



Dip. Informatica
University of Pisa

- Note that K-nn makes the local approximation to the target function for each new example to be predicted:

The computational cost is deferred to the **prediction phase!**

Moreover: high retrieval cost:

- Computationally intensive (in time): for each new input compute the distances from the test sample *to all* stored vectors
 - The time is proportional to the number of stored patterns
 - “ad-hoc” proximity search algorithms to optimize
 - E.g. by indexing the patterns
- Cost in space (all the TR data are stored)

Limitations: Curse of dimensionality



Dip. Informatica
University of Pisa

- When we have a lot of input variables (high dimension n), K-nn methods often fail because of the *curse of dimensionality* [*#ML*]
 - when the dimensionality n increases, the volume (*side* n) of the space increases so fast that the available data become sparse (*low sampling density*)
 - i.e. the amount of data needed to support the result often grows exponentially with the dimensionality.
- *Irrelevant features: The "curse of noisy"*
 - if the target depends on only few of many features in $\mathbf{x}$ (e.g. 2 out 20), we could retrieve a "similar pattern" with the similarity dominated by the large number of irrelevant features

It grows with the dimensionality
- Ergo a serious *generalization capability* issues

Distance based methods: a consideration and inductive bias



Dip. Informatica
University of Pisa

- ***Distance* or *metric* based** approaches:
- Common line among e.g. K-NN and kernel-based approaches (and K-means, see later):
- What is ***similar***? Measure when couple of patterns are *similar*.
- Giving a metric (e.g. the Euclidian metric was assumed so far) K-nn poses a relevant ***bias*** on the solution (much more than the alg. details)
- The metric is typically domain dependent
 - E.g. two strings in a language, in biology, etc.
 - Based on the prior knowledge of the domain expert
 - In some way, the same of *feature engineering* to represent the problem
- Looking forward: is it possible to learn the metric? Yes, NN [#ML], Learning Kernels, ...

K-means

Unsupervised learning

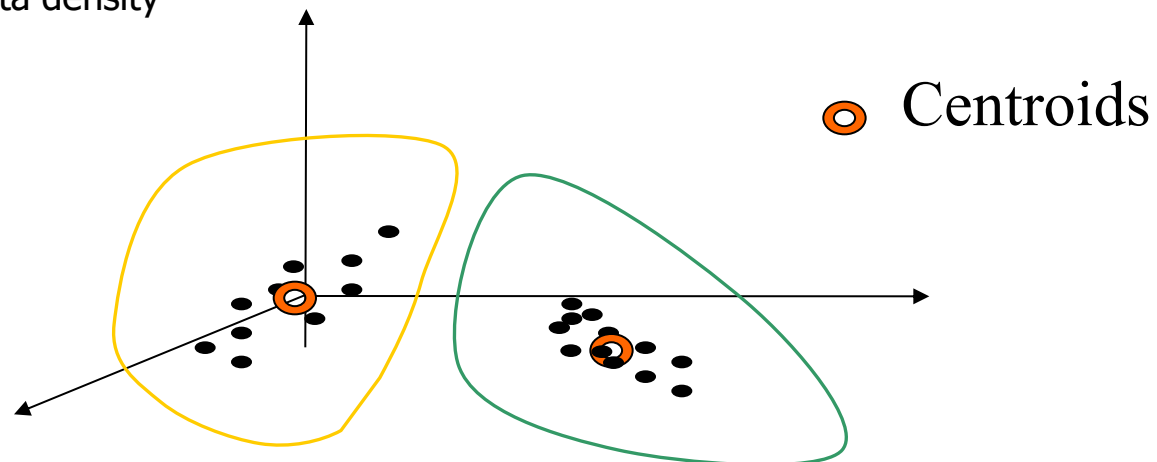
K-means and others

Tasks: Unsupervised Learning

- **Unsupervised Learning: No teacher!**

TR = set of unlabeled data $\langle \mathbf{x} \rangle$

- E.g. to find *natural groupings* in a set of data
 - Clustering
 - Dimensionality reduction/ Visualization/Preprocessing
 - Modeling the data density



Clustering:

Partition of data into clusters (subsets of “similar” data)

Utility

In nature:

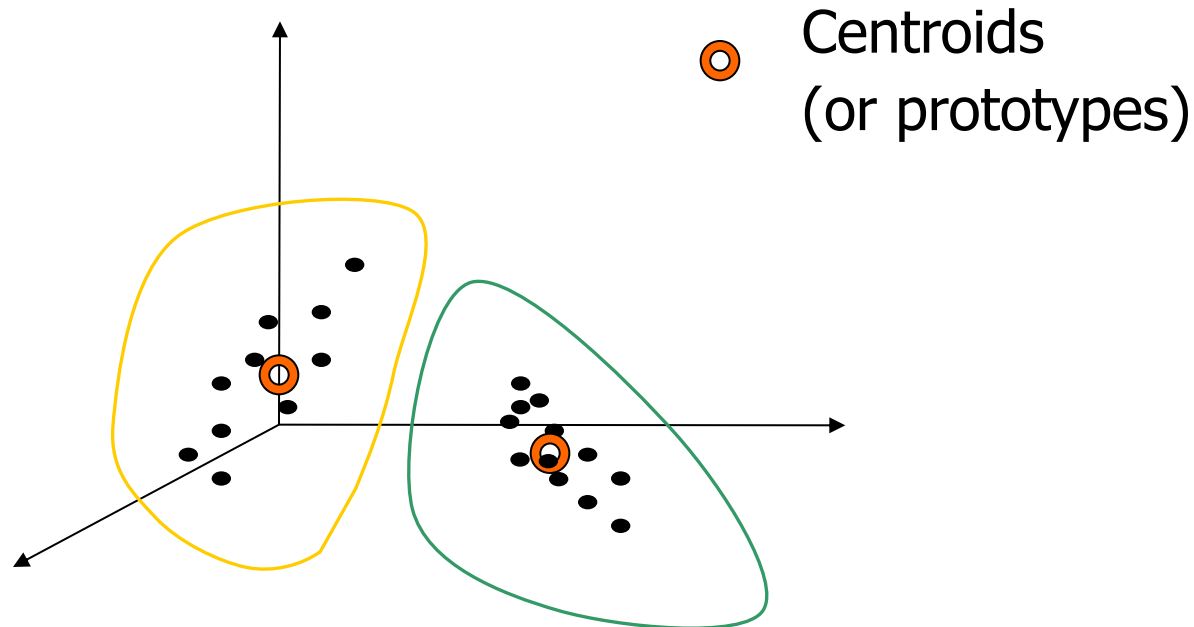
- Example (Taxonomy): organize life forms: what are mammals ?
- Example (human learning): babies learn to recognize ("cluster") familiar faces before they can understand human speech

In **Data Analysis**:

- Explorative data analysis
 - Discover data latent structure automatically
 - Find unknown patterns
- Preprocessing for other ML approaches
 - E.g. by dimensionality reduction
- Moreover: Unlabeled data is much cheaper to obtain
 - Examples: e.g. 10 million images (sampled frames from YouTube) with state-of-the-art results on object recognition (22,000 categories)

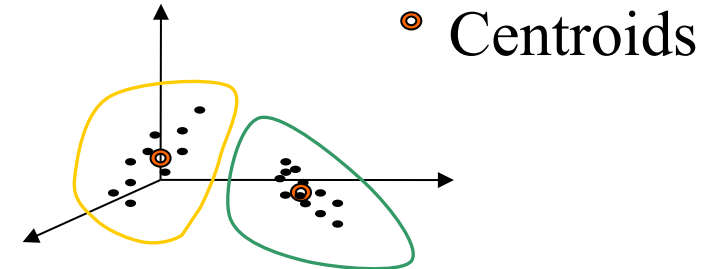
Clustering

Def Partition of data into clusters (subsets of “similar” data): patterns within a valid cluster are more similar to each other than they are to a pattern belonging to a different cluster



H space for clustering (#tech)

- **Goal:** optimal partitioning of unknown distribution in $\mathbf{x}$ -space into regions (clusters) approximated by a cluster center or *prototype*.



- **H:** a set of vector quantizers $\mathbf{x} \rightarrow c(\mathbf{x})$ (that represent the cluster)
continuous space $\rightarrow$ discrete space
- Example of Loss function: measures the vector quantizer optimality
- A common **loss function** would be the **squared error distortion**:

Def

$$L(h(\mathbf{x}_p)) = \|\mathbf{x}_p - c(\mathbf{x}_p)\|^2$$

- The average value over the distribution of inputs is the average distortion or reconstruction or quantization error

K-means

- The k -means is the simplest and most commonly used algorithm employing a *squared error criterion*
- The k -means algorithm is popular because it is easy to implement, and its in general efficient
- However, it has several drawbacks and *limitations* that we will see after the algorithm presentation....

Basic K-means Batch Alg.



Dip. Informatica
University of Pisa

LBG(Linde, Buzo, Gray) or generalized Lloyd algorithm

Un algoritmo di *ricerca locale*

Def

- (1) Choose k cluster centers to coincide with k randomly-chosen patterns or k randomly defined points inside the hypervolume containing the pattern set.
- (2) Assign each pattern to the closest cluster center (the winner).
- (3) Recompute the cluster centers (geometric centroid, i.e. mean) using the current cluster memberships.
- (4) If a convergence criterion is not met, go to step 2. E.g.
 - no (or minimal) reassignment of patterns to new cluster centers,
 - minimal decrease in squared error.

Details

- Given the cluster centers $c_1 \dots c_K$ (vectors of dim n , as $\mathbf{x}$)
- (2) For each $\mathbf{x}$ the winner is (the nearest prototype):

Def

$$i^*(\mathbf{x}) = \arg \min_i \|\mathbf{x} - c_i\|^2 \rightarrow \text{Norm 2}$$

Euclidian distance

$$\|\mathbf{x} - c_i\|^2 = \sum_{j \rightarrow n} (x_j - c_{ij})^2$$

- Now $\mathbf{x}$ belong to cluster i^*

- (3) For each cluster i the new mean (centroid) is:

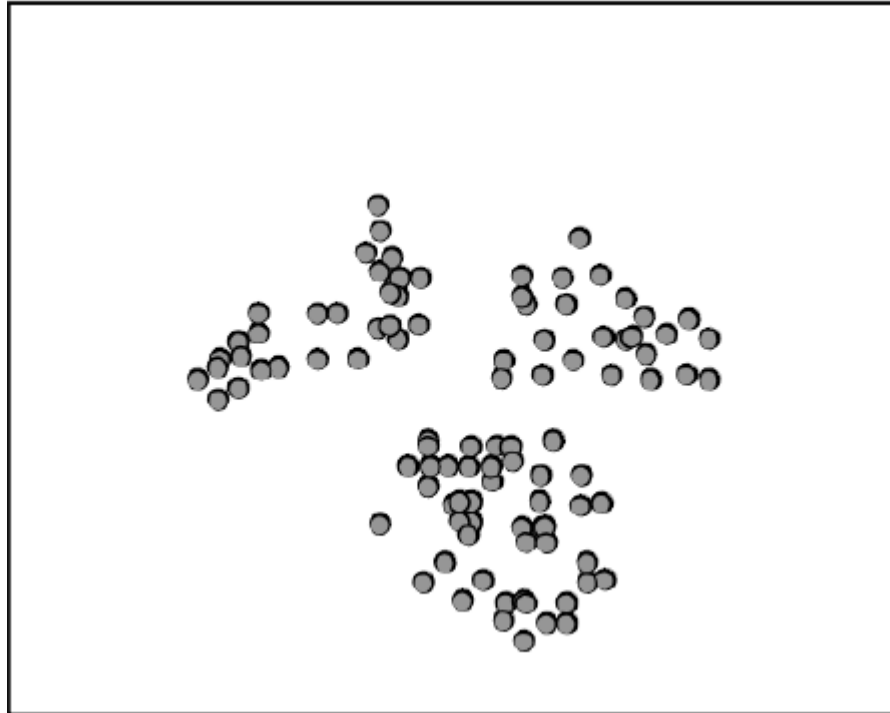
Def

$$c_i = \frac{1}{|\text{cluster}_i|} \sum_{j: \mathbf{x}_j \in \text{cluster}_i} \mathbf{x}_j$$

$\swarrow$ Vector $\searrow$ Number of cluster members $\searrow$ Vectors

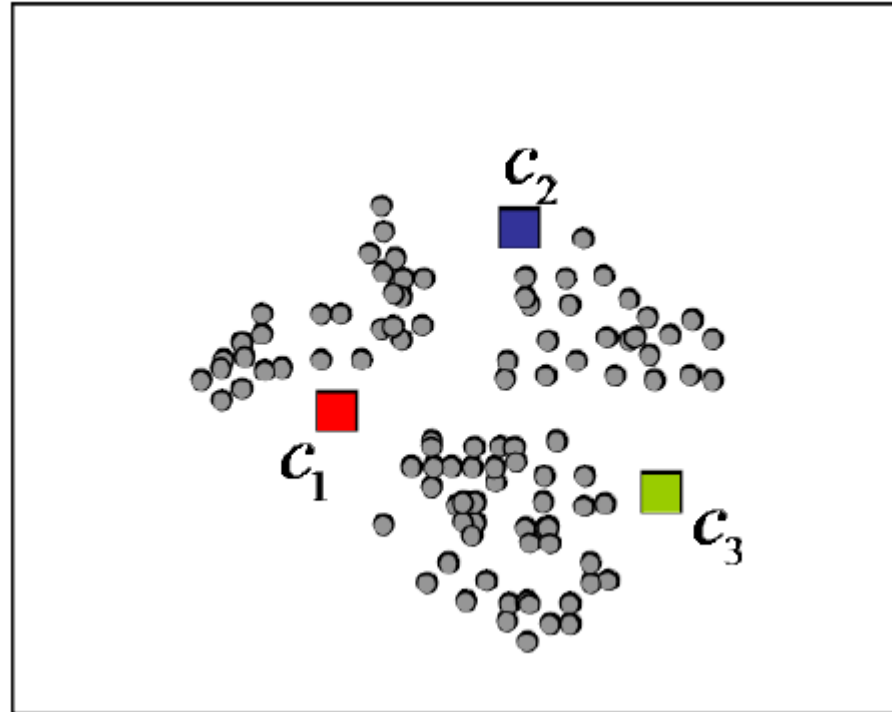
*Note: $\|\cdot\|^2$ is the square of norm 2,
hence the sum of squares (no root)*

K-means example (1)



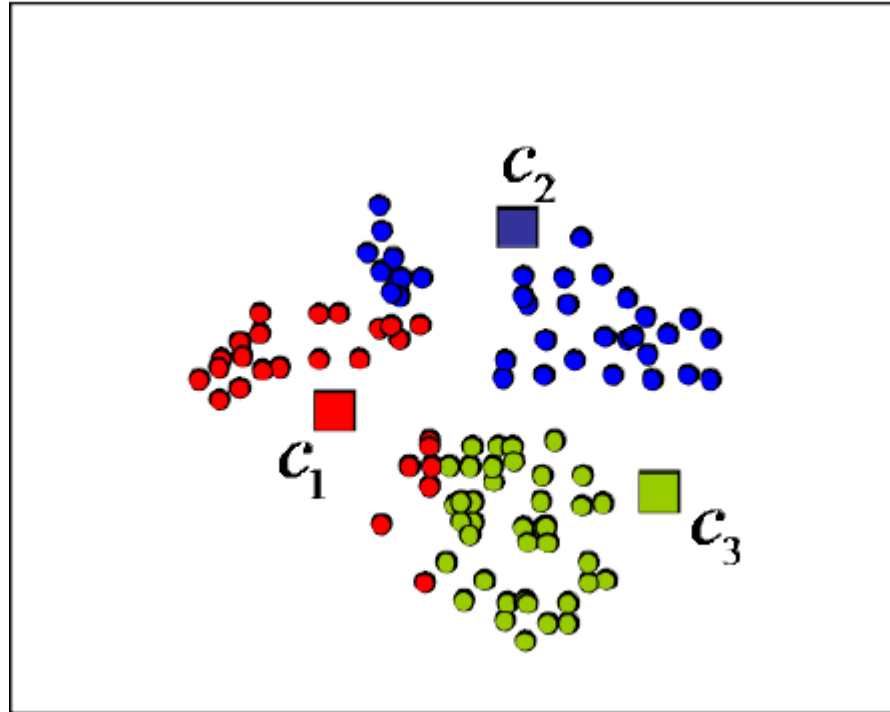
Input data

K-means example (2)



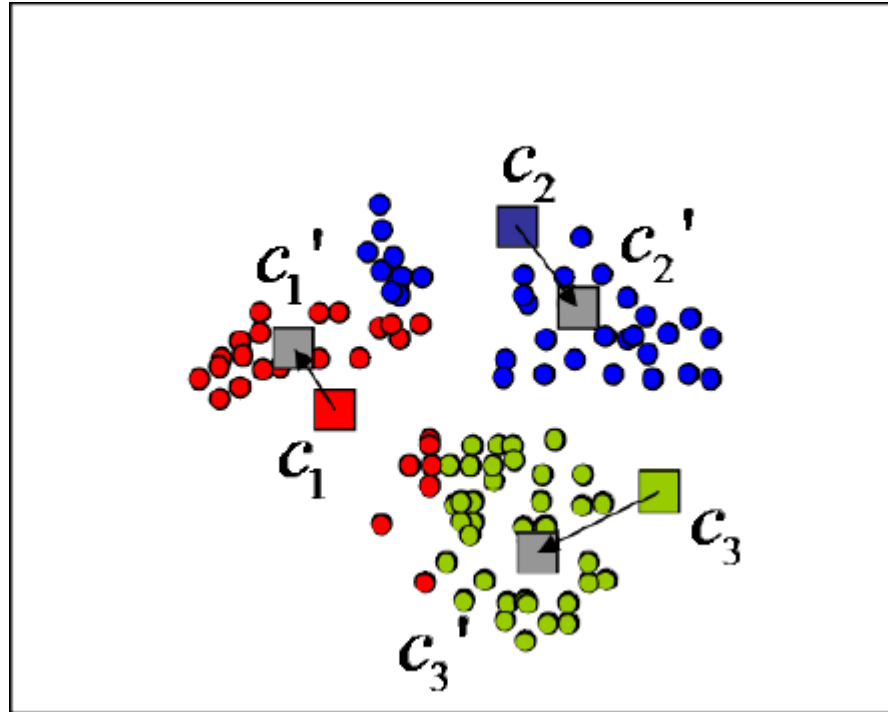
(1) Initialize $K = 3$ prototypes

K-means example (3)



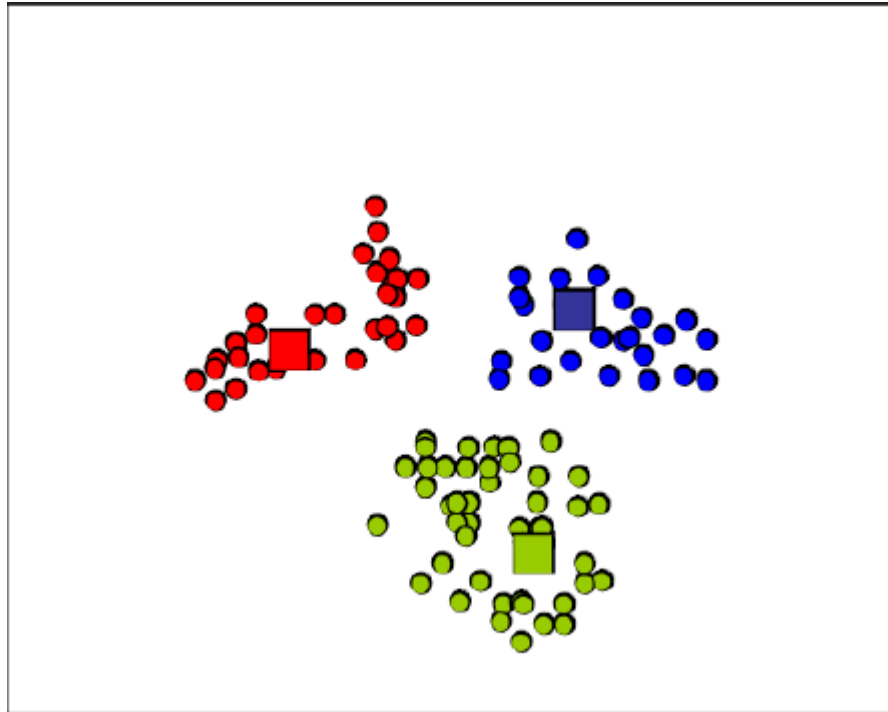
(2) Assign samples to the nearest prototype/cluster

K-means example (4)



(3) Compute the updated prototypes C'

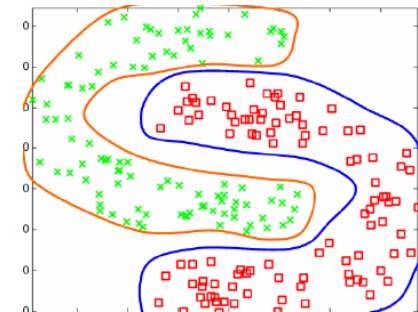
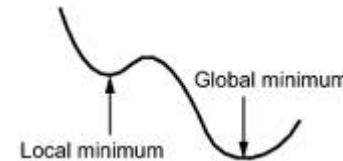
K-means example (5)



(2) Update cluster assignment

K-means limitations

- The number of clusters to find must be provided
 - Trials and-error to find the most suitable K
- Local minima of loss $L(\mathbf{x})$ make the method dependent on initialization
 - Run multiple times from different random initializations
 - Initialize with a heuristic
- It can work well for compact and hyperspherical clusters.
 - (image what happens for cluster with other forms)
- No visualization properties
 - K-means does not allow us to project data in a lower dim space



Evaluation (just few words)

- How is the output of a clustering algorithm evaluated?
- What characterizes a 'good' clustering result and a 'poor' one?
- Subjective: little in the way of 'gold standards' exist in clustering except in well-prescribed subdomains.
- Objective measures (no treated here): e.g. quantization error (but note that depends on the number of centroids K)!
- But not "trivially" the label of an underlining classification task, called *purity* (entropy, mutual inf., ...) (else you can use a classifier)!
- Quality of model/clustering versus quality of results (domain dependent)

Other topics in Unsupervised

- Hierarchical Clustering → *dendrograms*
- Principal Components, Curves and Surfaces
- Association Rules [DM]
- Independent Component Analysis and Exploratory Projection Pursuit
- Multidimensional Scaling
- ART (adaptive resonance theory) NN
- Self-organizing map (SOM) neural networks [#ML]
- ...

Other approaches for preprocessing

Among the multitude of approaches, it deserves to be mentioned in this course:

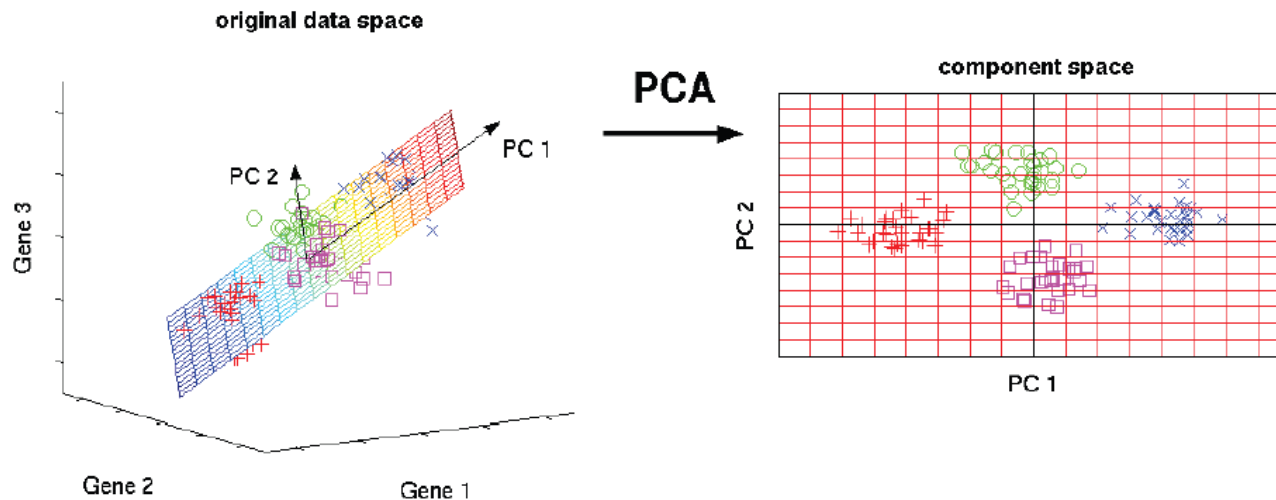
- **Dimensionality reduction** (in unsup. Learning)

Def

- $\langle X_1, X_2, \dots, X_n \rangle \rightarrow \langle X'_1, X'_2, \dots, X'_{n'} \rangle \quad n > n'$
- where the new features can be a combination of the original ones.

- E.g. PCA Principal Component Analysis (#tech)

- New axes are computed along the directions of max variance of the original manifold of data (eigenvectors)



Other approaches for preprocessing



Dip. Informatica
University of Pisa

- **Feature selection**

- Where we **choose a subset** of all the features
- Domain knowledge (*features engineering*)
- *Automatic features selection* on the basis of their redundancy or relevance in the task (the most informative)
- Many approaches: The problem is difficult as the learning problem one.
- But useful to reduce noise and as a result in itself (explanations).

- **Outlier detection**

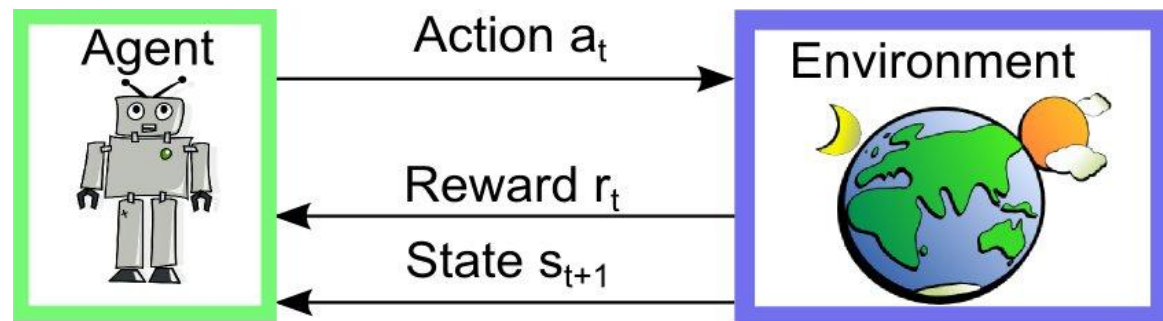
- Find unusual data values that are not consistent with most observations (e.g. due to abnormal measurements errors)

Other tasks in ML....

Other tasks:

Reinforcement Learning

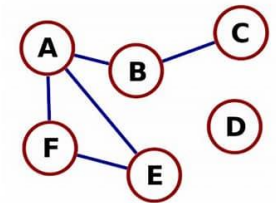
- **Reinforcement Learning** (learning with right/wrong critic).
 - Adaptation in autonomous systems (e.g. in robotics)
 - “the algorithm learns a policy of how to act given an observation of the world. Every **action** has some impact in the environment, and the environment provides feedback that guides the learning algorithm”.
 - Instead of supervision *for each step* we have information about wins/losses (**rewards** or **punishment**) for the final state
 - Actions (policy) need to maximize the amount of received rewards
 - The learning decides which actions were most responsible for wins/losses



Reinforcement Learning Setup

Other tasks

- **Semi-supervised learning**
 - combines both labeled and unlabeled examples (typically in greater number) to generate an appropriate function or classifier.
- **Learn to rank:** (e.g. for search engines)
 - When the input is a set of objects and the desired output is a ranking of those objects
- **On-line/continuous learning:** new examples/tasks are learned in time
- **Structured domain learning and relational learning [#ML]:**
 - The input and/or output domain can be structured in the form of **sequences** (signals, time series, ...)
 - Or even more complex: trees, **graphs, networks**:
 - Vector patterns + relationships with other components
 - Structured Input: Molecules, biological networks, web pages as graph/network data, social networks, street maps, brain modeling, knowledge graphs, ...
 - Structured Output: network data in social analysis, parse tree from sentences, ...

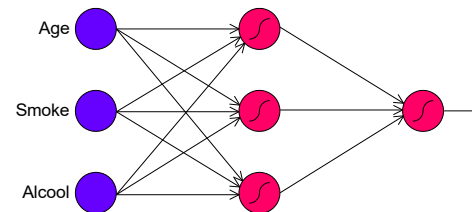
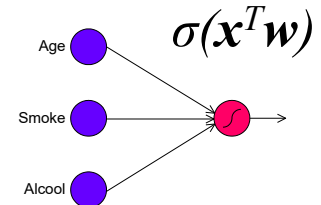


Other models

Neural Networks: a paradigm for both supervised and unsupervised learning (#ML)

Close to our view of Linear Threshold Unit (LTU):

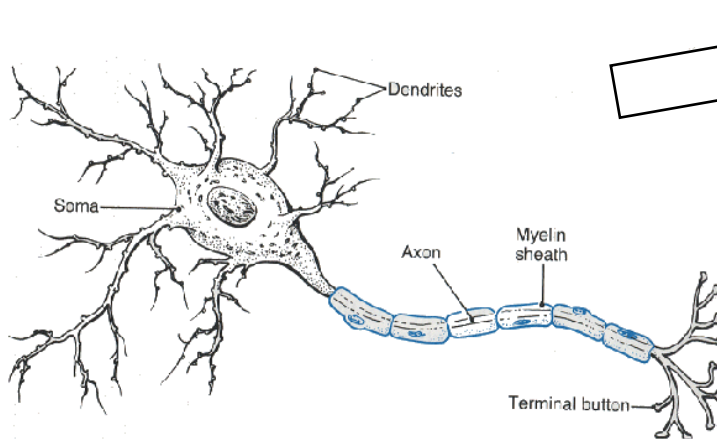
- Indeed, LTU is close to the basic unit of an Artificial NN, *the perceptron*.
- A NN is a *network* of such *non-linear* units
 - with universal approximation capability
- Composing a very flexible approach for any purpose of the ML.
 - Gradient descent approach for learning (backproagation of error from output layer back to the input layer)



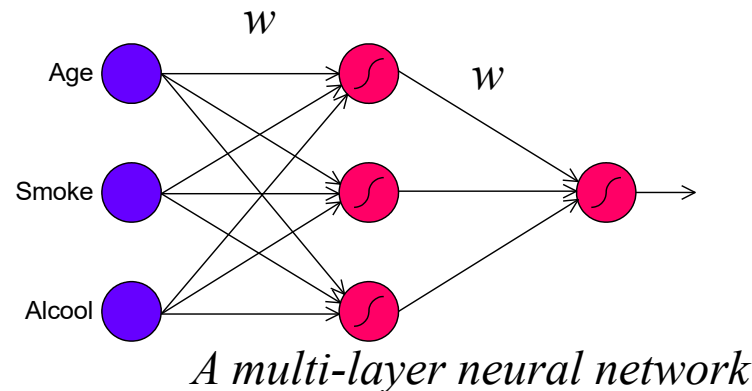
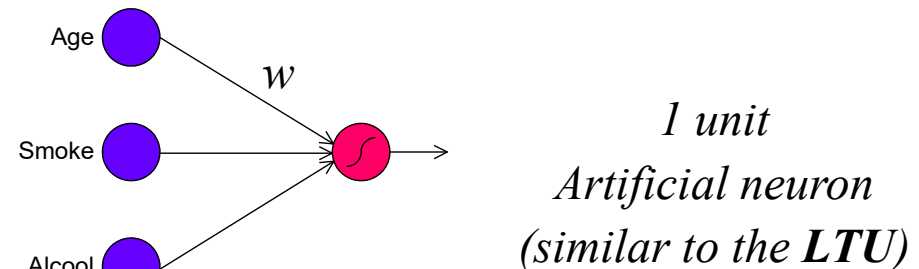
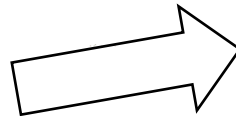
Neural Networks (#ML)

Natural inspired models: **Neural Networks**

- Studied as computing paradigms since the 40`s
- Neurobiological inspiration
- Nowadays: set of powerful computing models for *function approximation* and with predictive capabilities supported by a rigorous theoretical ground (*learning theory*)



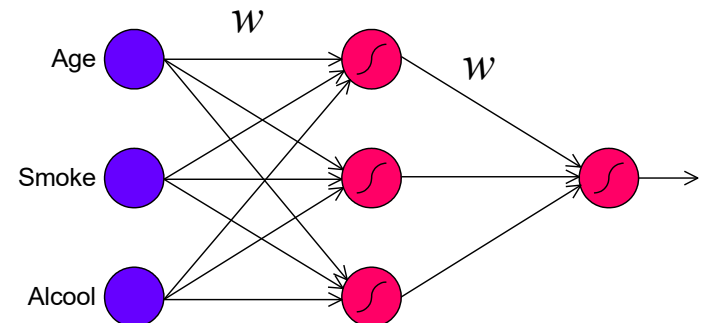
Biological neuron



Why internal layers?

The internal (*hidden*) layer with non-linear units provides the NN with

- The capability to extract by learning a *new representation of data* (learning abstract features)
- The new representations simplify the classification task at the last layer
- Non-linear in $\mathbf{w}$ adaptive *basis expansion* where the ϕ are learned (depend on $\mathbf{w}$) ! (but unfortunately it also leads to a non-linear optimization problem)
- Distributed representation (versus symbolic): similarity can be more easily treated by real values vectors
- Very flexible approach for non-linear tasks
- For both classification and regression tasks
- Universal approximation capability



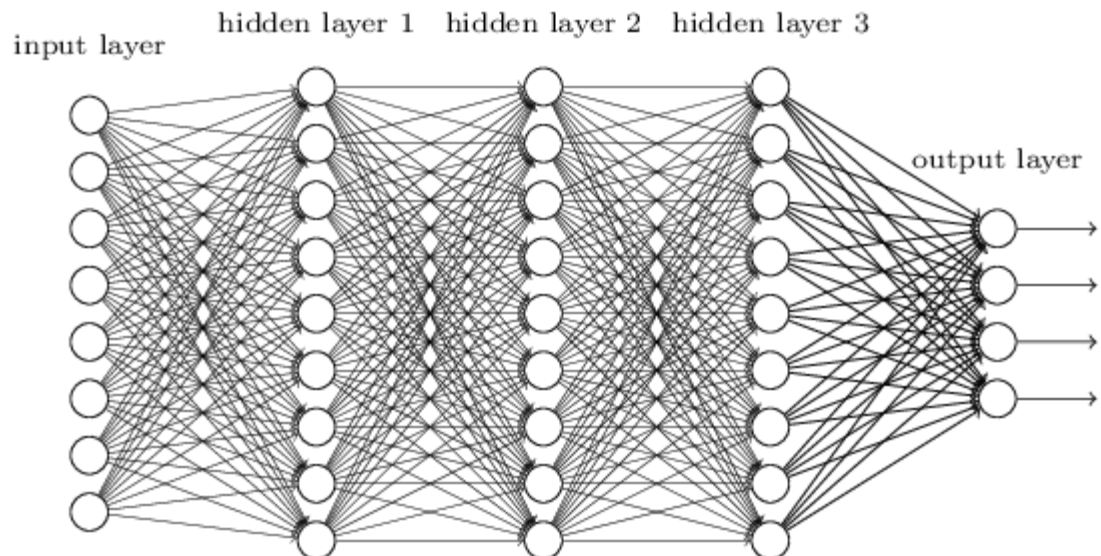
A multi-layer neural network

Deep Learning (DL)

- E.g. by deep multilayer NN architectures
- Common aspects among different approaches:
 - Multiple layers of nonlinear processing units
 - Supervised or unsupervised learning of *feature representations* in each layer, with the layers forming a *hierarchy* from low-level to high-level features/representations (*different levels of abstraction*).

Do you remember *deep learning* for the **Go** game competition?
(end of lecture on games)

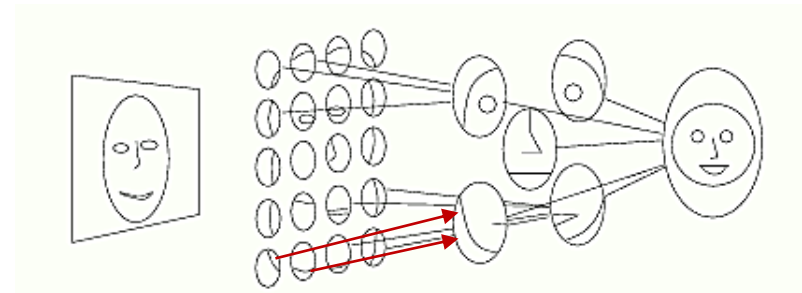
Or face recognition (lect 1 of ML intro)



Learning representations of data

- *Representation learning* versus the *feature engineering* approach (see slide on Distance based methods)
- Automatically discover the representations (a.k.a. the features) needed for detection or classification
- Increase level of **abstraction** through different layers

Example: An observation (e.g., an image) can be represented in many ways such as a vector of intensity values per pixel, or in a more **abstract** way as a set of edges, regions of particular shape, etc.



- Edges form motifs, motifs assemble into parts (e.g. corner), parts form specific objects (e.g. eyes),

- Performs best when input information has some form of structure, e.g. spatial, temporal, ... (images, language, music, ...)

An other example (with CNN)

From pixels to object identity:

Deep learning resolves this difficulty by breaking the desired complicated mapping into a series of nested simple mappings, each described by a different layer of the model.

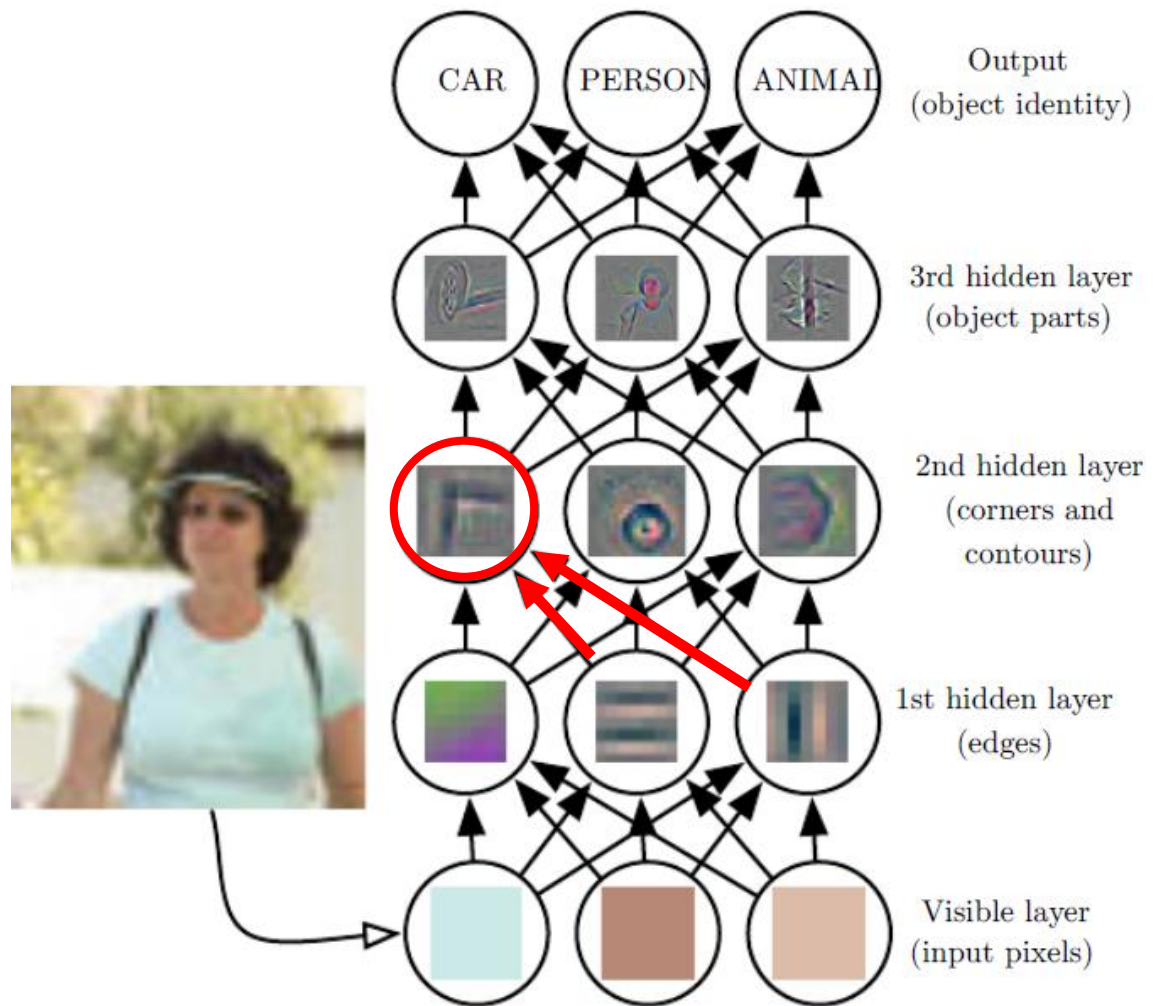
A series of *Hidden layers* extracts increasingly abstract features from the image.

Note: The new representation simplify the classification task at the last layer.

Biological visual system analogies

The images here are visualizations of the kind of feature (pattern) represented by each hidden unit

A. Micheli



Combine to generalize

- And the abstract features can be “reused” at the higher levels.
 - It can combine them to generalize to unseen (during training) cases:
 - Woman without glasses
 - Man with glass
- Woman with glass can be easily represented as well

Internal representations can be combined

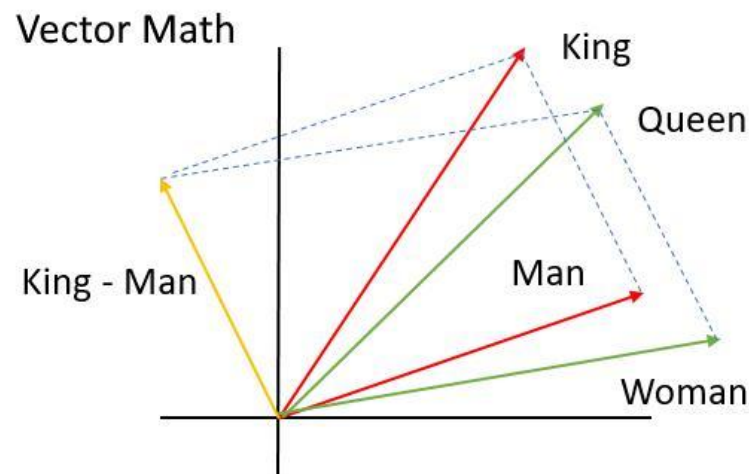


Here operations on
‘**vector representations** learned by model’
and then used to generate new images

Combine to generalize (NLP)

Representation (Rep) in the vector space developed by the Deep model (inner layers)

- $\text{Rep}(\text{king}) - \text{Rep}(\text{male}) + \text{Rep}(\text{female}) = \text{Rep}(\text{queen})$ [gender relationship]
 - In this case, the vector difference between *king* and *male* captures the concept of monarchy
- $\text{Rep}(\text{Paris}) - \text{Rep}(\text{France}) + \text{Rep}(\text{Poland}) = \text{Rep}(\text{Warsaw})$ [concept of capital]
 - In this case, the vector difference between *Paris* and *France* captures the concept of capital city



DL advantages

- Exploit **compositionality** of internal representations:
 - An exponential gain in representational power,
 - due to the fact that simpler concepts represented in a layer of the network
 - can be *exploited as primitives* many times *by the next layer* to represent more complex concepts,
 - avoiding explicit *combinatorial* representation (and learning) of features
 - E.g. you have to learn each word/sub-pattern but not directly all the possible combinations (i.e. without having to see all the configurations), hence:
 - *Less examples* are needed to reach good generalization capability!

DL results

- Deep networks were difficult to be trained in the past,
- but combining
 - improvement in the *techniques for training* of such huge models,
 - *HPC* (e.g. GPU) and
 - *large data collections* from industry and real applications (e.g. millions of images)
- they nowadays work very well making a *revolution in the AI* approach to **real-world solutions**
- "*These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains*"

LeCun, Y., Bengio, Y., & Hinton, G. (May 2015). *Deep learning*. Nature, 521(7553), 436-444

DL results: a “historical” example: Image to text

Image to text applications

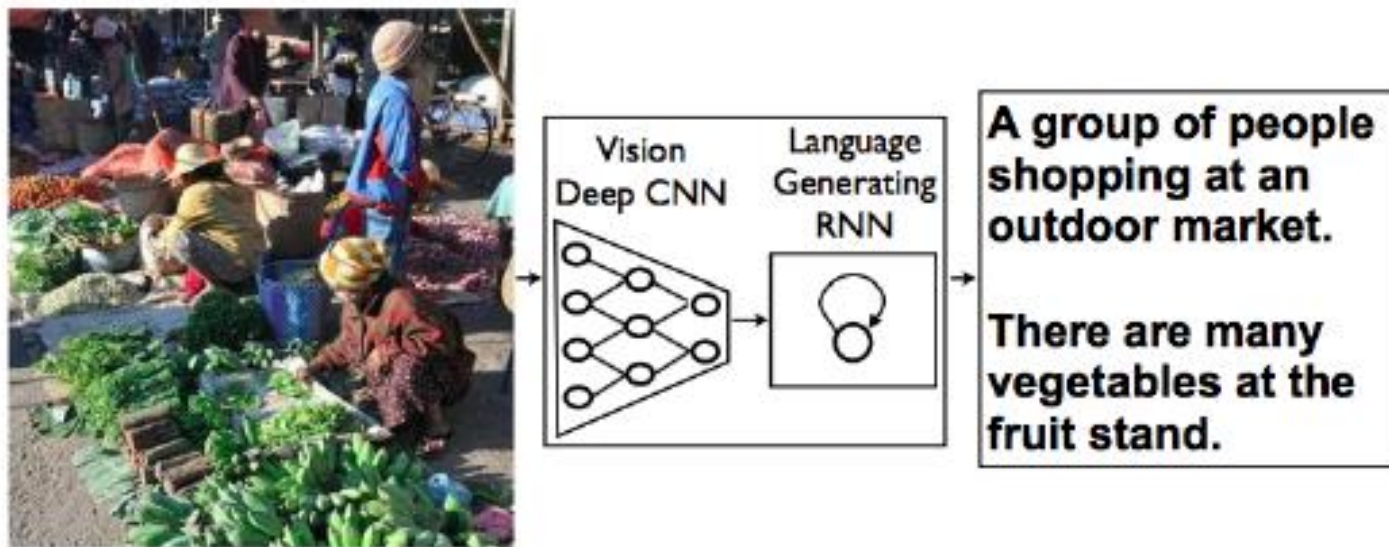
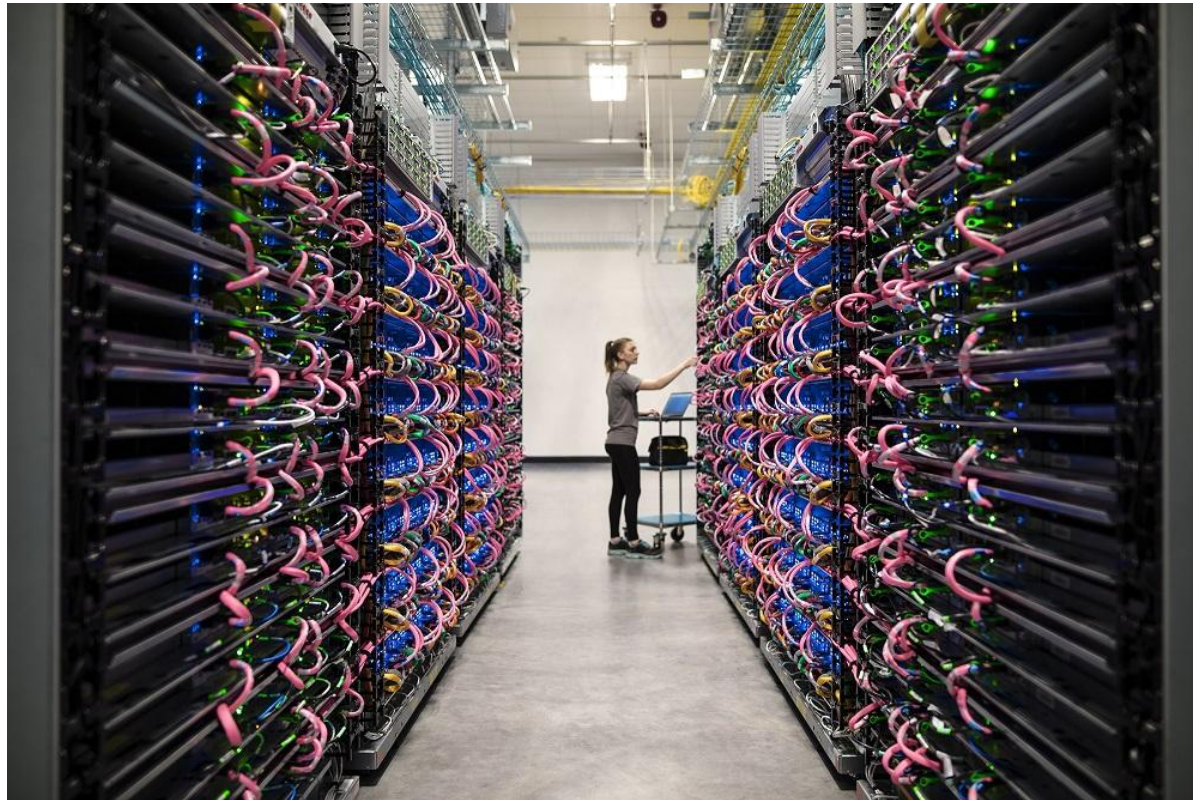


image taken from: Vinyals et al., *IEEE TPAMI*, 2016

More in the #ML courses

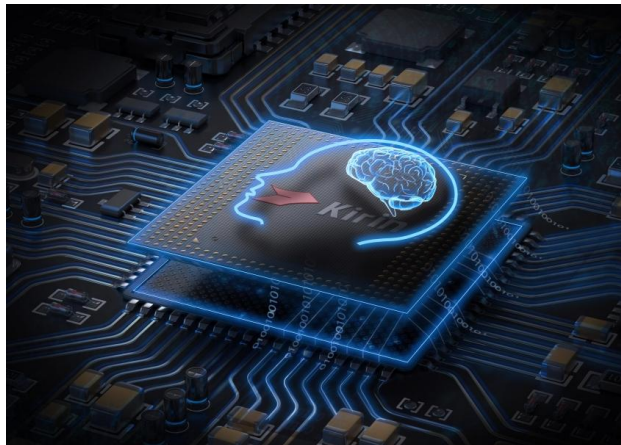
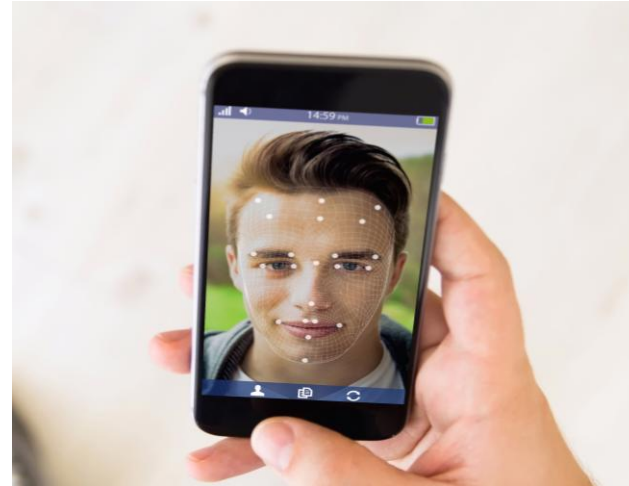
TPU @ Google



A server rack containing multiple **Tensor Processing Units**, a special-purpose chip designed specifically for machine learning

The original TPU was designed specifically to work best with Google's TensorFlow. Just for inference (mapping) !!!!

In your everyday life...



Bibliography

- K-NN
 - Mitchell: K-nn: **chap 8**
 - AIMA, ed. 3: chap 18.8, **ed. 4:** in **19.7**
- K-means:
 - these slides or any text that describe it (also references in Wikipedia etc.).
- Other cited methods (e.g. Neural Networks): general bibliography of the course, and master degree courses
- And AIMA ed. 4: chap. 21 (ma va oltre)

DRAFT, please do not circulate!

For information

<https://pages.di.unipi.it/micheli>

Alessio Micheli

micheli@di.unipi.it

<https://ciml.di.unipi.it/>



Dipartimento di Informatica
Università di Pisa - Italy



**Computational Intelligence &
Machine Learning Group**